

maquette-scad

Parametric CAD in Typst, via OpenSCAD + Manifold

github.com/bernsteining/maquette · bernsteining.github.io/maquette



Version 0.1.0 · September 17, 2026

CONTENTS

Contents

Introduction	3
Where to find sample .scad files	4
Quickstart	4
Building geometry from Typst — scadypst	5
DSL reference	6
3D primitives	6
2D primitives (extrusion sources)	6
2D → 3D lifting (and 3D → 2D)	6
Transforms (2D or 3D)	6
Booleans and hulls	6
Plane operations	6
Colours and alpha	8
scadypst() — the compile call	9
Name clashes with Typst built-ins	10
Compiling .scad sources — compile-scad	11
Inspection: scadypst-info / compile-scad-info	12
Decomposing: scadypst-parts / compile-scad-parts	13
Vertex reduction: simplify(child, epsilon)	14
Smooth shading: calculate-normals(child, sharp_angle)	15
Two ways to add smooth normals	15
Ray casting: scadypst-raycast / compile-scad-raycast	16
Direct 2D → SVG: scadypst-svg / compile-scad-svg	17
Multi-file projects: compile-scad-tree	18
Very large assemblies	19
What compile-scad accepts	20

Introduction

maquette-scad produces a PLY mesh (or 2D SVG) from a geometry description. Two ways to write the description:

- **scadypst(tree)** — build geometry from Typst-native helpers (cube, sphere, difference, translate, ...) composed with Typst's own `for`, `range`, and `calc`. Returns PLY bytes.
- **compile-scad(source)** — pass the text of an existing `.scad` file. Returns PLY bytes.

Both entry points also have a `-svg` sibling (`scadypst-svg`, `compile-scad-svg`) for direct 2D vector output — no rasterizer in the loop.

This document is the plugin's Typst API reference: what each helper accepts, what the compile calls return, and how to route sidecar assets. Downstream rendering options (camera, lighting, shadows, tone mapping) belong to `maquette` — see its manual. For the OpenSCAD language itself see the OpenSCAD Users Manual.

Where to find sample .scad files

The DSL examples in this document build their geometry inline, so no external source is needed. For the compile-`scad(read("..."))` path, .scad sources are easy to find:

- `openscad/openscad examples/` — the set that ships with the OpenSCAD editor.
- BOSL2 — a large utility library with hundreds of documented fragments.
- Thingiverse (`openscad` tag) — community archive; many entries ship the .scad alongside the .stl.

Quickstart

`render-scad` compiles and renders in one call — pass it a DSL tree or a .scad path, plus any of `maquette`'s render options.

```
#import "@preview/maquette-scad:0.1.0": cube, difference, render-scad, sphere

#render-scad(
  difference(
    cube(20, center: true),
    sphere(12, fn: 48),
  ),
  camera: (40, 40, 40),
  up: (0, 0, 1),
)
```

`render-scad` also takes source text — same call, a .scad file instead of a tree:

```
#import "@preview/maquette-scad:0.1.0": render-scad

#render-scad(read("model.scad"))
```

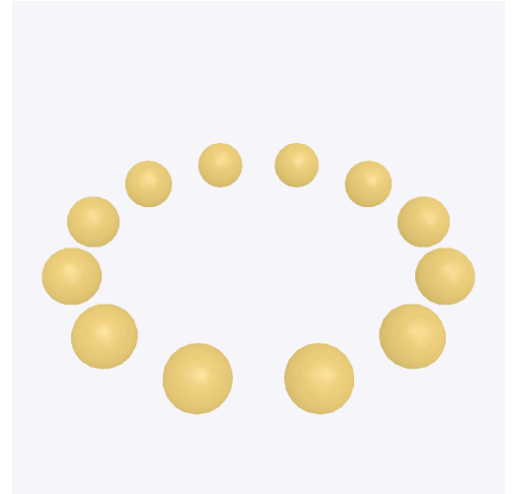
One call compiles and renders; no `maquette` import. Examples below use a `show-part` wrapper so the code can focus on geometry.

Building geometry from Typst — scadypst

`scadypst(tree)` walks a tree of Typst dicts and returns PLY bytes. Each helper (`cube`, `sphere`, `translate`, ...) builds a node; nothing runs until `scadypst` receives the whole tree.

Because the tree is Typst code, iteration and arithmetic come from Typst. A `..for` spread inside `union` places twelve spheres on a ring:

```
#let ring = union(..for i in range(12) {  
  let a = i * 30  
  let r = 20  
  (translate(  
    (calc.cos(a * 1deg) * r, calc.sin(a * 1deg) * r, 0),  
    sphere(3, fn: 24),  
  ),)  
})  
#show-part(scadypst(ring))
```



Variadic ops (`union`, `difference`, `hull`, `intersection`) accept `..items` — spread a Typst for block to feed a computed list.

DSL reference

Every helper takes Typst-native named arguments (`cube(20, center: true)`). Signatures below list positional args first, then named. `fn: N` sets the per-primitive segment count.

3D primitives

- **cube(size, center: false)** — size is a number (uniform) or a 3-array (`x, y, z`).
- **sphere(r, fn: none)**
- **cylinder(h, r: none, r1: none, r2: none, center: false, fn: none)** — pass `r` for a straight cylinder, `r1 + r2` for a cone / frustum.
- **polyhedron(points, faces)** — raw mesh: `points = ((x, y, z), ...)`, `faces = ((i, j, k, ...), ...)`.

2D primitives (extrusion sources)

- **square(size, center: false), circle(r, fn: none), ellipse(w, h, fn: none).**
- **polygon(points, paths: none)** — arbitrary 2D shape. `paths` is a list of index-rings into `points`: first ring is the outer boundary, rest are holes.
- **ngon(sides, r, fn: none)** — regular polygon.
- **star(points, outer, inner)** — n-pointed star.
- **rounded-square(w, h, r, fn: none)** — rectangle with radius-`r` corners.
- **scad-text(str, size: 10)** — glyph outlines from the font passed to `scadypst()`'s `font: param`.
- **import-mesh(file)** — reference an STL/OBJ passed via `scadypst()`'s `bin: dict`.

2D → 3D lifting (and 3D → 2D)

- **linear-extrude(h, child, center: false, twist: 0, scale: 1, slices: none)** — extrude a 2D shape to a height `h`. `twist` is degrees over the full height; `scale` is a taper factor (0..1) or an (`sx, sy`) pair.
- **rotate-extrude(child, angle: 360, fn: none)** — revolve a 2D profile (living in the +x half-plane) around the z-axis.
- **projection(child)** — flatten a 3D solid to its 2D shadow on the `Z=0` plane.

Transforms (2D or 3D)

- **translate(v, child)** — `v` is a 2- or 3-vector.
- **rotate(deg, child)** — Euler degrees, (`x, y, z`) or a single number (2D rotation).
- **scale(v, child)** — per-axis scale.
- **mirror(v, child)** — reflect across a plane whose normal is `v`.
- **multmatrix(m, child)** — 4×4 (or 4×3) affine matrix; escape hatch for anything the named transforms can't do.
- **resize(v, child)** — non-uniform scale to fit a target bounding box.
- **offset(d, child)** — grow (`d > 0`) or shrink (`d < 0`) a 2D shape by `d`.
- **color(rgb, child, alpha: none)** — RGB is a 3-array of 0..1 floats, or a 4-array (`r, g, b, a`). See the Colours + alpha section below.

Booleans and hulls

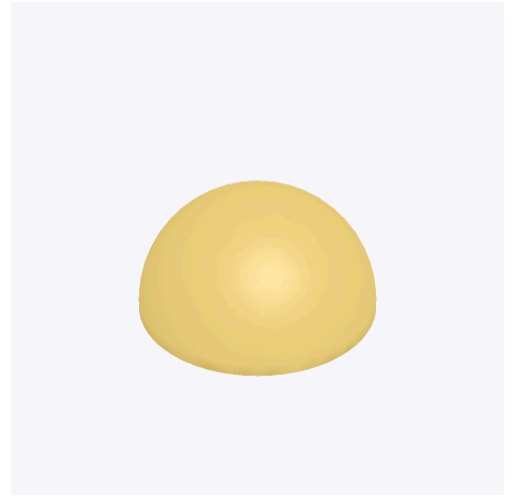
- **union(..items), difference(..items)** (first minus the rest), **intersection(..items).**
- **hull(..items)** — convex hull of the union (3D).
- **hull-pts(points)** — convex hull from a raw 3D point set (a list of (`x, y, z`) triples). Complements `hull()` when you have coordinates, not geometry, to wrap.
- **minkowski(..items)** — Minkowski sum (3D).

All variadic — use `..` spread with a Typst `for` block to feed a computed list of children.

Plane operations

- **slice(child, z: 0)** — horizontal cross-section of a 3D solid at the given `Z`. Returns a 2D shape; pipe into `scadypst-svg(..)` for a vector contour, or into further 2D ops. Distinct from `projection(..)`, which unions every horizontal slice.
- **trim(child, normal, offset: 0)** — cut with a plane. Keeps the half where `dot(pos, normal) ≥ offset`. Cheaper and exact vs the “difference() with a giant cube” trick.
- **simplify(child, epsilon: 0.01)** — Douglas-Peucker vertex reduction. Works on both 2D and 3D. See its own section for a before/after count.
- **calculate-normals(child, sharp_angle: 60)** — attach per-vertex normals for smooth shading, with a crease-angle threshold that keeps sharp edges crisp. 3D only. Renders show the effect only under shading: “smooth”.

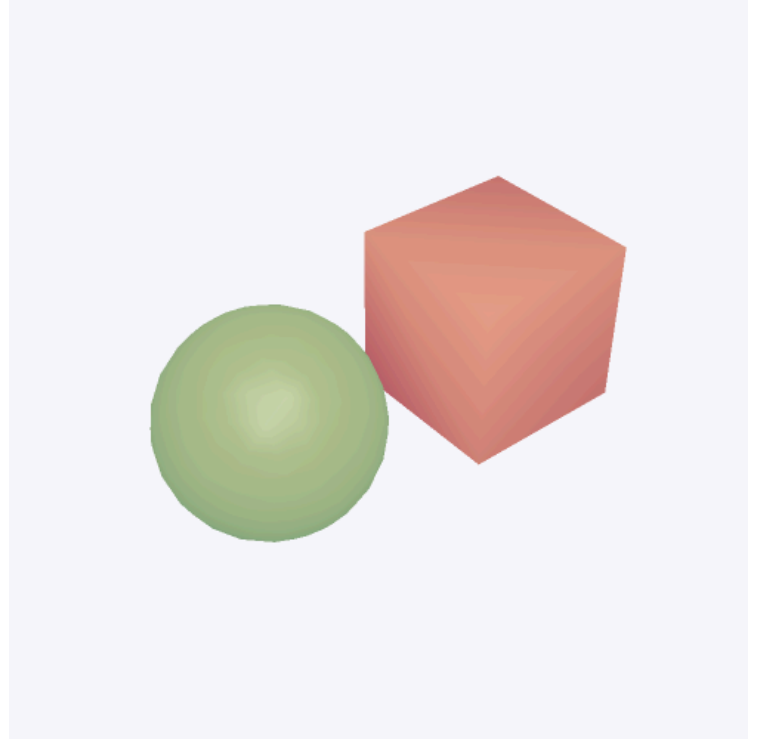
```
#show-part(scadypst(  
  trim(sphere(10, fn: 64), (0, 0, 1))  
))
```



Colours and alpha

Colours propagate through boolean operations. The emitted PLY carries per-vertex RGB and render-ply picks it up automatically. Any part without a `color()` wrapper inherits the render config's `color:` argument.

```
#show-part(  
  scadypst(union(  
    color((0.9, 0.3, 0.3), cube(10, center: true)),  
    color((0.3, 0.7, 0.4), translate((15, 0, 0), sphere(6, fn:  
32))),  
  )),  
  color: none,  
)
```



Alpha is supported. Two equivalent forms:

```
color((0.9, 0.35, 0.35, 0.5), child)           // RGBA in one 4-array  
color((0.9, 0.35, 0.35), child, alpha: 0.5)    // RGB + separate alpha
```

Interior geometry shows through translucent subtrees, so a coloured cover can reveal the shape's internal features.

scadypst() — the compile call

```
scadypst(node, bin: (:), font: none, fn: 32)
```

- **node** — the tree returned by any DSL helper. Usually the top-level union / difference / a single primitive.
- **bin** — dict of sidecar bytes referenced by `import-mesh(file)`. Keys match the string passed to `import-mesh`, values are `read("path", encoding: none)`.
- **font** — TTF/OTF bytes for `scad-text(...)`. One font per compile.
- **fn** — default `$fn` for the whole compile. Per-primitive `fn`: overrides.

Returns the same PLY bytes that `compile-scad` returns — pass it to `render-ply`.

Name clashes with Typst built-ins

A glob-import (`#import "..": *`) shadows seven of Typst's own functions:

`scale`, `rotate`, `circle`, `square`, `ellipse`, `polygon`, `color`. (Additionally, the OpenSCAD text primitive is re-exported as `scad-text` to avoid shadowing `#text`.)

Two ways around it:

```
// Option 1 – namespace the plugin
#import "@preview/maquette-scad:0.1.0"
#let part = maquette-scad.scadypst(
  maquette-scad.difference(
    maquette-scad.cube(20, center: true),
    maquette-scad.sphere(12, fn: 48),
  )
)

// Option 2 – glob-import the plugin's `scad-*` aliases
#import "@preview/maquette-scad:0.1.0": scad-color, scad-scale, scad-rotate, scad-circle, scad-square,
scad-ellipse, scad-polygon
```

Both leave Typst's own `#text` / `#circle` / ... intact. The `scad-*` aliases live alongside the unprefix names, so you can mix approaches within one document.

Compiling .scad sources — compile-scad

For a standalone .scad file, one call is enough:

```
#render-scad(read("part.scad"))
```

Real-world .scad files rarely stand alone: they use / include a library, import an STL or OBJ for a fastener, or ship a font for embossed text. The wasm sandbox has no filesystem, so every sidecar is passed explicitly as bytes:

```
#compile-scad(read("main.scad"),
  files: (
    "utils.scad":      read("utils.scad"),
    "gears.scad":      read("gears.scad"),
    "MCAD/nuts_and_bolts.scad": read("MCAD/nuts_and_bolts.scad"),
  ),
  bin: (
    "mount.stl":      read("mount.stl", encoding: none),
  ),
  font: read("logo.ttf", encoding: none),
  fn: 48,
  trace: "trace.log",
)
```

Options:

- **source** (positional string) — the .scad text. Almost always read("path.scad").
- **files** (dict) — resolves every use <name> / include <name> in the source. Keys are the exact names the .scad uses; nested paths like MCAD/foo.scad are keys with slashes.
- **bin** (dict) — sidecar mesh files referenced by import(). Keys match the import argument. Recognised formats: STL and OBJ (DXF, 3MF, and AMF are not).
- **font** (bytes) — a TTF or OTF file used by text(). One font per compile.
- **fn** (int) — default \$fn for this compile. Any per-primitive fn: overrides it.
- **trace** (string) — dump the evaluator trace to this path. Handy when a nested for or module isn't producing what you expected.

Inspection: scadypst-info / compile-scad-info

Returns a Typst dict with the final geometry's stats without building the PLY. Useful for laying parts out by their real size, annotating a document with computed volumes, or failing early if a compile produced empty geometry.

Fields: bbox_min, bbox_max, center, radius, volume, surface_area, num_tri, num_vert, genus.

```
#let info = scadypst-info(cube(20, center: true))
#raw(block: true, lang: "yaml",
  "volume:      " + str(info.volume) + " mm³\n" +
  "surface_area: " + str(info.surface_area) + " mm²\n" +
  "num_tri:      " + str(info.num_tri) + "\n" +
  "bbox_min:     " + repr(info.bbox_min) + "\n" +
  "bbox_max:     " + repr(info.bbox_max)
)
```

```
volume:      8000 mm³
surface_area: 2400 mm²
num_tri:      12
bbox_min:     (-10, -10, -10)
bbox_max:     (10, 10, 10)
```

compile-scad-info(src, files: (:), bin: (:), font: none, fn: 32) is the .scad-source variant with the same return shape.

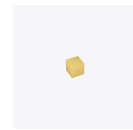
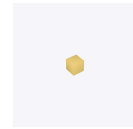
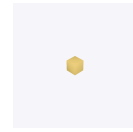
Decomposing: `scadypst-parts` / `compile-scad-parts`

Splits the final geometry into its connected components via `Manifold::decompose()` and returns each as its own PLY. The return type is `array<bytes>`; each element is renderable via `render-ply` in isolation.

Useful for laying out multiple pieces separately (laser-cut sheet layouts, exploded-view annotations, per-part BOM tables).

```
#let parts = scadypst-parts(union(
  translate((0, 0, 0), cube(4, center: true)),
  translate((10, 0, 0), cube(4, center: true)),
  translate((20, 0, 0), cube(4, center: true)),
))
Got #parts.len() parts:
#for p in parts {
  show-part(p, width: 25%)
  h(0.3em)
}
```

Got 3 parts:



`compile-scad-parts(src, files: (:), bin: (:), font: none, fn: 32)` is the `.scad-source` variant.

Vertex reduction: `simplify(child, epsilon)`

Collapses edges shorter than `epsilon` (in the input's units) — a Douglas-Peucker style pass that strips micro-detail from booleans or slims a mesh before shipping it through PLY / SVG.

Dimension-agnostic: pass a 2D shape (`CrossSection`) or a 3D solid (`Manifold`); the op dispatches on the child.

```
#let ring = union(..range(64).map(i => {
  let a = i * 360deg / 64
  translate((calc.cos(a) * 10, calc.sin(a) * 10),
    circle(1.2, fn: 8))
}))
#let ring-info = scadypst-info(ring)
#let simple-info = scadypst-info(simplify(ring, epsilon: 0.5))
#raw(block: true, lang: "yaml",
  "before → num_tri: " + str(ring-info.num_tri) + "\n" +
  "after → num_tri: " + str(simple-info.num_tri)
)
```

```
before → num_tri: 1152
after → num_tri: 112
```

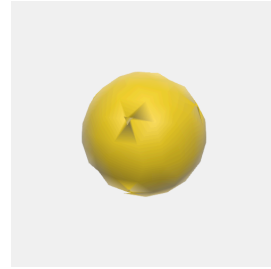
`epsilon` is in model units — tiny for a fine mesh, generous when you want to knock a curve down to a coarse polyline.

Smooth shading: `calculate-normals(child, sharp_angle)`

Attaches per-vertex normals to a 3D solid so maquette can smooth-shade curved surfaces while keeping crisp edges. Adjacent faces whose dihedral angle exceeds `sharp_angle` (degrees) stay sharp; the rest blend. The op must run AFTER final booleans/hulls — those discard any existing normals.

The effect only shows under smooth shading. `openscad-view` is flat by design, so switch shading: `"smooth"` (and drop `openscad-view`) for the render call.

```
#let part = calculate-normals(
  difference(sphere(10, fn: 24), cube(12, center: true)),
  sharp_angle: 30,
)
#render-ply(scadypst(part),
  width: 3.5cm, shading: "smooth",
  azimuth: 30, elevation: 25)
```



Lower `sharp_angle` = more edges classed as sharp (crisp box corners). Higher = more edges smoothed (softer curves).

Two ways to add smooth normals

`calculate-normals` (above) is a **tree op** — it smooths whatever subtree it wraps. Use it when only one region of a bigger model should render smooth (say the sphere but not the cube parts).

The end-call helpers `scadypst(..., smooth-normals: 30)` and `compile-scad(..., smooth-normals: 30)` are the **whole-scene** shortcut: they run the same `calculate_normals` pass on the final mesh right before it's serialized. Same output, less typing when you want the entire model smoothed:

```
// tree-op form — smooths just the sphere:
scadypst(union(
  calculate-normals(sphere(10, fn: 24), sharp_angle: 30),
  translate((15, 0, 0), cube(6, center: true)),
))

// end-call form — smooths everything:
scadypst(my-model, smooth-normals: 30)
compile-scad(read("part.scad"), smooth-normals: 30)
```

Either way, the render call needs a smooth shading mode (`shading: "smooth"`) to actually consume the normals.

Ray casting: scadypst-raycast / compile-scad-raycast

Fires a segment from origin to end at the evaluated geometry and returns an array of hits sorted by distance:

```
(
  ( face_id: <int>, distance: <float>,
    position: (x, y, z), normal: (x, y, z) ),
  ..
)
```

Use it to sample a terrain height under an XY coordinate, pick the face at a screen click, or check line-of-sight between two points. Empty array means the segment missed.

```
#let part = union(
  cube(10, center: true),
  translate((0, 0, 12), sphere(5, fn: 32)),
)
#let hits = scadypst-raycast(part,
  (0, 0, 30), (0, 0, -30))
#raw(block: true, lang: "yaml",
  "hits: " + str(hits.len()) + "\n" +
  hits.map(h =>
    " d=" + str(calc.round(h.distance, digits: 2)) +
    " z=" + str(calc.round(h.position.at(2), digits: 2))
  ).join("\n")
)
```

```
hits: 4
d=0.22 z=17
d=0.38 z=7
d=0.42 z=5
d=0.58 z=-5
```

compile-scad-raycast(src, origin, end, files: (:), bin: (:), font: none, fn: 32) is the .scad-source variant.

Direct 2D → SVG: `scadypst-svg` / `compile-scad-svg`

For sources that resolve to a 2D shape (circle, square, polygon, text, hulls of 2D things, offset, slice, projection), the plugin can emit an SVG document directly — no maquette in the loop, no rasterizer. The result is resolution-independent, Inkscape-editable, and laser-cutter-ready.

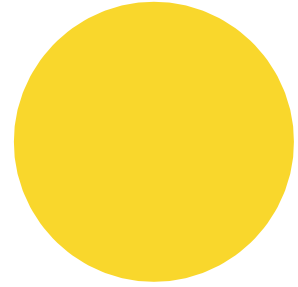
Errors if the tree resolves to a 3D solid.

```
#image(scadypst-svg(difference(
  union(
    circle(10, fn: 64),
    translate((14, 0), square(20, center: true)),
  ),
  circle(4, fn: 48),
)), width: 65%)
```



`slice(child, z: 0)` composes cleanly with `-svg`: 3D model → horizontal cross-section → vector contour, in one call.

```
#image(scadypst-svg(slice(sphere(10, fn: 64), z: 4)), width: 60%)
```



`compile-scad-svg(src, ...)` is the `.scad-source` variant.

Multi-file projects: compile-scad-tree

A project split across many files with use `<.> / include <.>` compiles in one call — `compile-scad-tree` reads the entry file and follows its include graph for you, so there's no file list to maintain:

```
#import "@preview/maquette-scad:0.1.0": openscad-view, render-scad-tree

#render-scad-tree(
  "Cyclone.scad",
  root: "cyclone-src/",
  read: p => read(p),
  fn: 8,
  ..openscad-view,
  azimuth: 35,
  elevation: 20,
  up: (0, 0, 1),
)
```

`root` is the source folder joined onto each discovered path; `entry` is the top file inside it. Pass a `read`: lambda scoped to your document — `read: p => read(p)` — so the sources are read from **your** project (a package's own `read()` can't reach your files). The whole Cyclone-PCB-Factory below — 69 files — comes from that single call:

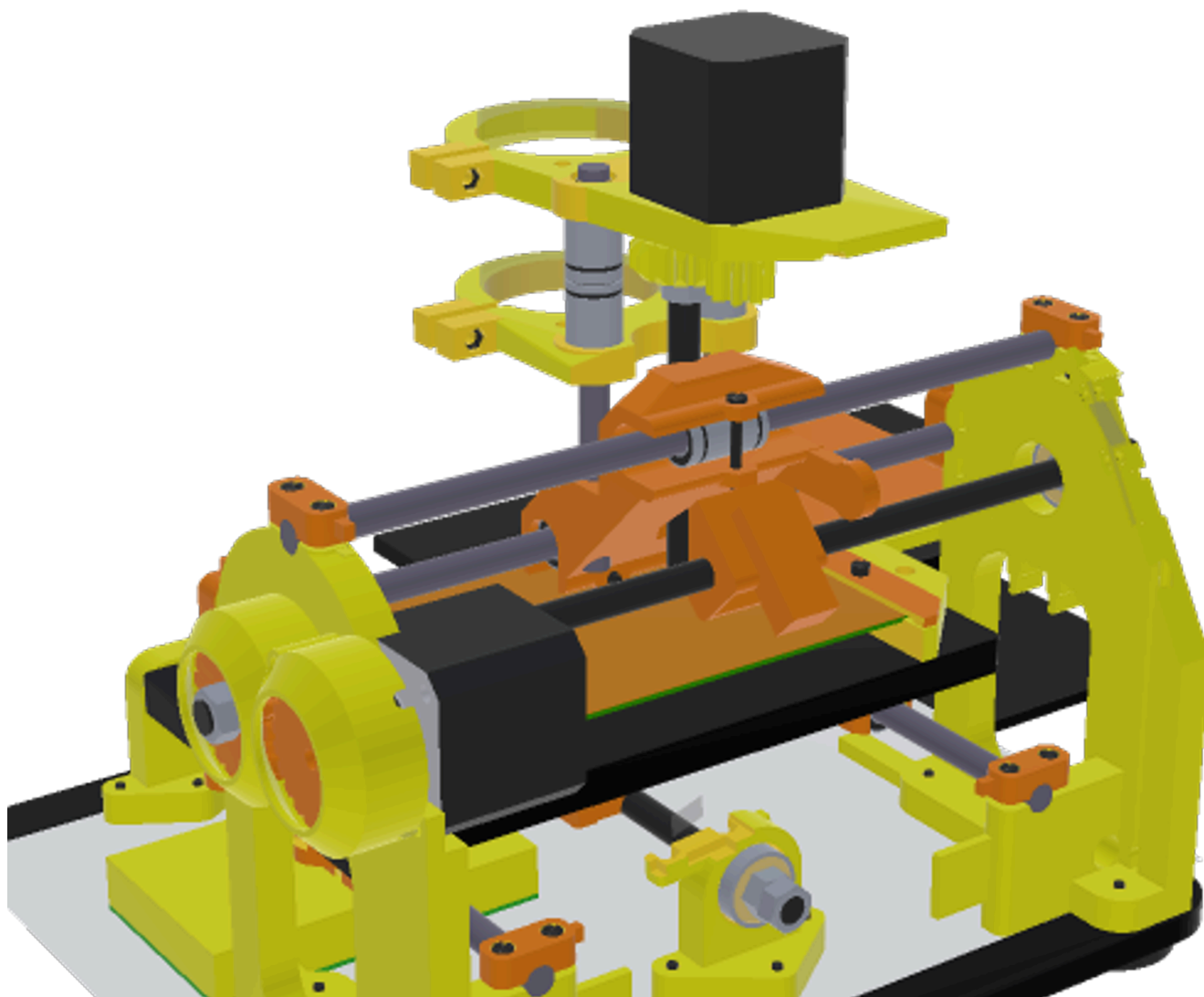


Figure 1: The full Cyclone-PCB-Factory from one `compile-scad-tree` call.

Need the files as a dict instead (to edit one before compiling)? `scad-collect(entry, root:)` returns `path → source` for `compile-scad(main, files: ...)`.

Very large assemblies

If a model is too big to build in the plugin, compile it once natively and render the resulting .ply:

```
#import "@preview/maquette:0.1.3": render-ply

#let full = read("model.ply", encoding: none)
#render-ply(full, ..openscad-view, azimuth: 35, elevation: 20)
```

What compile-scad accepts

If your `.scad` source works in the OpenSCAD editor, it should work here — the evaluator covers everything a typical file uses. Known gaps: `surface()` (heightmap import), `import()` of DXF, `$vpr` / `$vpt` / `$vpd` / `$vpf` viewport variables, and adaptive tessellation from `$fa` / `$fs` (we use `$fn` or the per-primitive `fn:` instead). Any external STL / OBJ mesh referenced by `import()` must be routed through the `bin:` argument since the plugin has no filesystem access.

The exhaustive per-feature tracker lives at `crates/maquette-scad/FIDELITY.md`.

Two wasm modules cooperate. `maquette-scad.wasm` (this crate) compiles geometry to PLY. `maquette.wasm` renders it. They exchange nothing but the PLY blob, so you can also feed a `scadypst(...)` output into `maquette-gltf` (wrapped in a minimal `.gltf`) when you want PBR shading on procedural geometry.